Name : Shivraj Vaibhav Jagtap
Roll No : EN24107060
Class : SY - AI\&DS
Subject : Python for Visual AI

Assignment 2:

Build a tool that analyses an image's histogram and automatically adjusts brightness and contrast for a more balanced result in Python using an image processing library. It has to accept the image as input from the user.

Step 1: Importing the necessary python libraries

In [1]:
```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt
from math import log10, sqrt
import skimage
from skimage.metrics import structural_similarity as ssim
import statistics
```

Step 2: Loading and reading the image that is to be converted into grayscale

In [2]:
```
img=cv.imread("lena2.jpg")
```

In [3]:
```
cv.imwrite("img.jpg",img)
img1= cv.imread("img.jpg")
```

Step 3: Converting the image into grayscale using various methods like,

1. Average Method (Arithmetic Mean)
2. Lightness Method
3. Luminosity (Perceptual) Method
4. Luma Method

In [4]:
```
img_f=img.astype(np.float64)
r,g,b=cv.split(img_f)
cv.imwrite("img_f.jpg",img_f)
avg=(r+g+b)/3
cv.imwrite("avg_img.jpg",avg)
img2= cv.imread("avg_img.jpg")
```

In [5]:
```
luminosity=(0.29912*r)+(0.587*g)+(0.114*b)
cv.imwrite("lumi_img.jpg",luminosity)
img3= cv.imread("lumi_img.jpg")
```

In [6]:
```
luma=(0.2126*r)+(0.715*g)+(0.0722*b)
cv.imwrite("luma_img.jpg",luma)
img4= cv.imread("luma_img.jpg")
```

In [7]:
```
lightness=(cv.max(r,g,b)+cv.min(r,g,b))/2
cv.imwrite("light_img.jpg",lightness)
img5= cv.imread("light_img.jpg")
```

Step 4: Subplotting all the grayscale converted images

In [8]:
```
fig, axs = plt.subplots(1, 5, figsize=(15,8))
axs[0].imshow(img1)
axs[0].set_title('Original Image')
axs[0].axis('off')
axs[1].imshow(img2)
axs[1].set_title('Average Method')
axs[1].axis('off')
axs[2].imshow(img3)
axs[2].set_title('Luminosity Method')
axs[2].axis('off')
axs[3].imshow(img4)
axs[3].set_title('Luma Method')
axs[3].axis('off')
axs[4].imshow(img5)
axs[4].set_title('Lightness Method')
axs[4].axis('off')
```

Out[8]:
```
(np.float64(-0.5), np.float64(799.5), np.float64(399.5), np.float64(-0.
5))
```



Step 5 : Comparing the factors like PSNR, SSIM & Entropy

In [9]:
```
def PSNR1 (I, I1):

    mse = np.mean((I - I1)**2)

    if mse == 0:
        return float('inf') #only possible if both the images are identic
    max_pixel = 255.0
    psnr = 20 * log10(max_pixel / sqrt(mse))
    return psnr

print("PSNR of Original and Average:\t",PSNR1(img1,img2))
print("PSNR of Original and Lumi:\t",PSNR1(img1,img3))
print("PSNR of Original and Luma:\t",PSNR1(img1,img4))
print("PSNR of Original and Light:\t",PSNR1(img1,img5))
```

```
PSNR of Original and Average:    29.082382605801712
PSNR of Original and Lumi:       29.05832888044936
PSNR of Original and Luma:       28.969966471090483
PSNR of Original and Light:      29.44053249120065
```

In [10]:
```
img_e1=skimage.measure.shannon_entropy(img_f)
print("The Entropy of Original Image is:\t\t",img_e1)
img_e2=skimage.measure.shannon_entropy(img2)
print("The Entropy of Grayscale (Avg) Image is:\t",img_e2)
img_e3=skimage.measure.shannon_entropy(img3)
print("The Entropy of Grayscale (Lumi) Image is:\t",img_e3)
img_e4=skimage.measure.shannon_entropy(img4)
print("The Entropy of Grayscale (Luma) Image is:\t",img_e4)
img_e5=skimage.measure.shannon_entropy(img5)
print("The Entropy of Grayscale (Light) Image is:\t",img_e5)
```

```
The Entropy of Original Image is:                7.435580175685837
The Entropy of Grayscale (Avg) Image is:         7.162231014303195
The Entropy of Grayscale (Lumi) Image is:        7.15375382153407
The Entropy of Grayscale (Luma) Image is:        7.127697569215105
The Entropy of Grayscale (Light) Image is:       7.12030871228442
```

In [11]:
```
dr=255.0

score1=ssim(img_f,img2,channel_axis=2,data_range=dr)
score2=ssim(img_f,img3,channel_axis=2,data_range=dr)
score3=ssim(img_f,img4,channel_axis=2,data_range=dr)
score4=ssim(img_f,img5,channel_axis=2,data_range=dr)
print("SSIM Value of avg_img is:\t",score1)
print("SSIM Value of lumi_img is:\t",score2)
print("SSIM Value of luma_img is:\t",score3)
print("SSIM Value of light_img is:\t",score4)
```

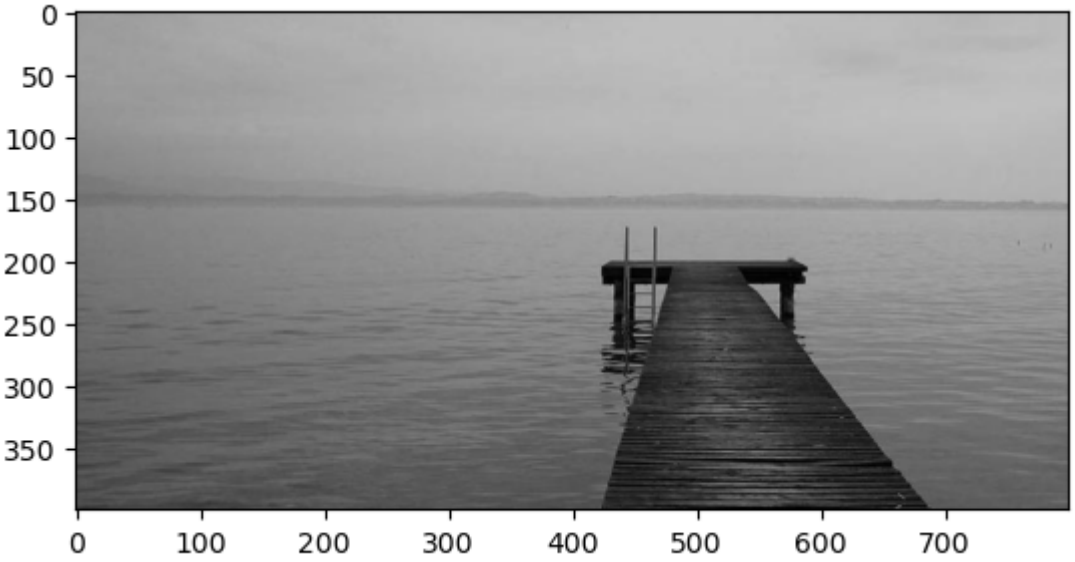
```
SSIM Value of avg_img is:        0.9818859470337086
SSIM Value of lumi_img is:       0.9805313784614323
SSIM Value of luma_img is:       0.9801142191561616
SSIM Value of light_img is:      0.9754030059361126
```

Step 6 :

Selecting the optimal method of grayscale conversion and fonding the minimum and maximum intensities of the pixels in the gryscale image

In [12]:
```
gray=cv.imread("lumi_img.jpg")
plt.imshow(gray)
min_gray=gray.min()
print("Min value of Gray image is:\t",min_gray)
max_gray=gray.max()
print("Max value of Gray image is:\t",max_gray)
```

```
Min value of Gray image is:	 0
Max value of Gray image is:	 193
```



In [13]:
```
o_img=cv.imread("img_f.jpg")
min_o=o_img.min()
print("Min value of Original image is:\t",min_o)
max_o=o_img.max()
print("Max value of Original image is:\t",max_o)
```

```
Min value of Original image is:	 0
Max value of Original image is:	 217
```

Step 7 : Linear Scaling Parameters

The scaling factor α (gain) and the offset β (bias) are calculated as:

$$\alpha = \frac{255}{\max_o - \min_o}$$

$$\beta = -\alpha \cdot \min_o$$

This is for stretching a gray image so it uses the full range from pure black (0) t

In [14]:
```
alpha=(255/(max_o-min_o))
beta=(-alpha)*min_o
```

Step 8 : Image Transformation

The final pixel values are mapped using the formula:

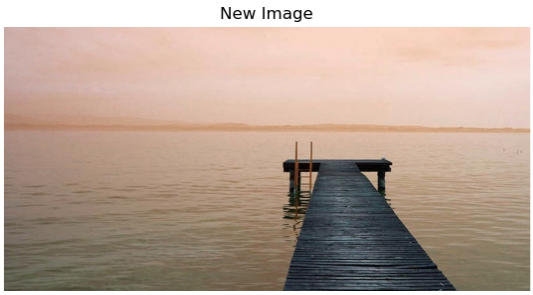
$$n_{\text{img}} = (\alpha \cdot o_{\text{img}}) + \beta$$

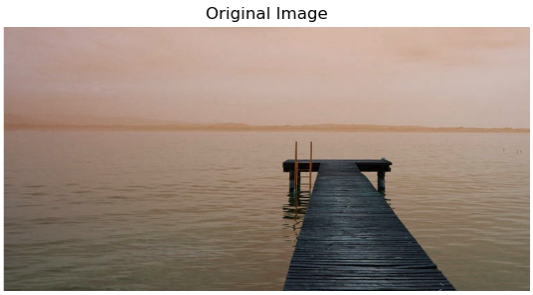
In [15]:
```
n_img=(alpha*o_img)+beta
cv.imwrite("new_img.jpg",n_img)
new_img=cv.imread("new_img.jpg")
```

Step 9 : Subplotting the original as well as the editted image

In [16]:
```
fig, axs = plt.subplots(1, 2, figsize=(15, 10))
axs[0].imshow(new_img)
axs[0].set_title('New Image')
axs[0].axis('off')
axs[1].imshow(o_img)
axs[1].set_title('Original Image')
axs[1].axis('off')
```

Out[16]:
```
(np.float64(-0.5), np.float64(799.5), np.float64(399.5), np.float64(-0.
5))
```





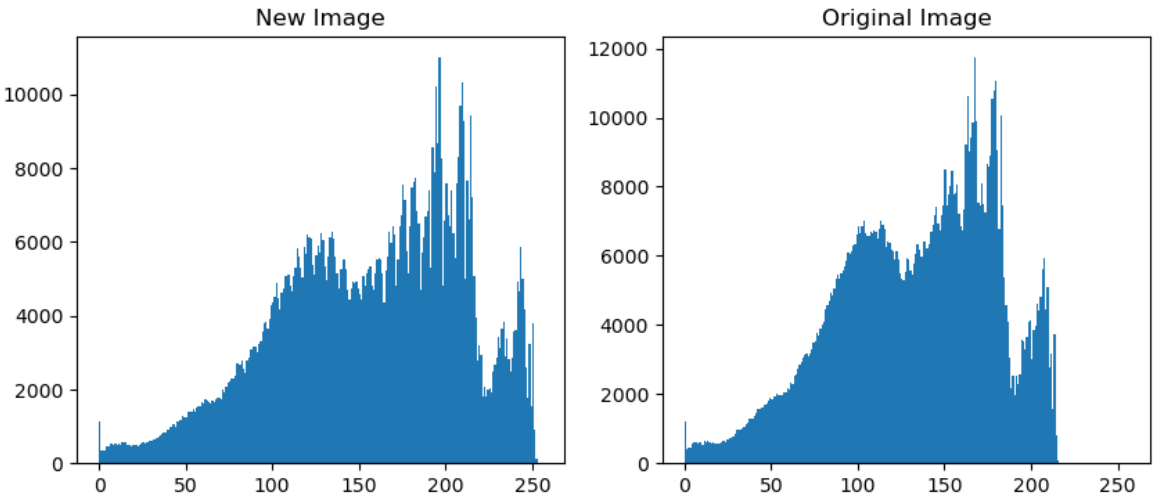
Step 10 : Subplotting the histograms of original as well as the editted image

In [17]:
```
# Original histogram
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.title("New Image")
plt.hist(new_img.ravel(), 256, range=[0,256])

# Adjusted image histogram
plt.subplot(1,2,2)
plt.title("Original Image")
plt.hist(o_img.ravel(), 256, range=[0,256])

plt.show()
```



Conclusion

The experiment successfully demonstrated the use of image histograms to analyze and enhance image quality. By examining the distribution of pixel intensities, brightness and contrast characteristics of an image were effectively understood.

> The implementation of automatic brightness and contrast adjustment using histogram stretching improved the visual appearance of the image by spreading intensity values across the full range (0-255).